

A Differential Evolution Approach to CTRNN Parameter Learning^{*}

De Falco Ivanoe¹, Della Cioppa Antonio², Donnarumma Francesco³,
Maisto Domenico¹, Prevete Roberto³, Tarantino Ernesto¹

¹ ICAR-CNR, Napoli.

² DIIE, Università di Salerno.

³ Università degli Studi di Napoli Federico II

Abstract. Recently, Continuous Time Recurrent Neural Networks (CTRNN) has kindled interest as agent control systems. Within this context, target behaviours can be achieved by finding suitable neural network parameters. Even though binary and real-valued Genetic Algorithms are usually deployed to search the parameter space of CTRNNs, they may fail to identify relevant solutions because of granularity, boundedness and blocking limitations. Differential Evolution algorithm, instead, may overcome these problems and, at the same time, preserves the advantages of the evolutionary approach.

1 Introduction

In recent years, within the autonomous agents community, there has been a growing interest in Continuous Time Recurrent Neural Networks (CTRNN) viewed as simple models of nervous systems for controlling the behaviour of agents. In this context the problem of achieving target agent behaviours is redefined as the problem of identifying suitable neural parameters, insofar as CTRNNs can approximate the trajectories of any dynamical system on a compact of $\mathbb{R}^n$.

Among different learning algorithms, Genetic Algorithms (GA) are usually deployed to perform searches in the parameter space of CTRNNs. Unfortunately, this approach sets a severe constraint on the parameters choice that GAs can produce, because the parameter values are both limited by the resolution of encoding (**granularity**) and confined in a priori fixed interval (**boundedness**).

To deal with these constraints, Yamauchi and Beer proposed a real-valued encoding for CTRNNs which improves the learning process because parameter values are allowed to be in $\mathbb{R}$. Nevertheless, in that case, the learning process may be slowed down and, more crucially, problems arise that prevent real-valued GAs (rvGA) from finding global optima (**blocking**). Trying to overcome some of the limitations mentioned above, we attempt a different approach based on a Differential Evolution (DE) algorithm. Differential Evolution (DE) is a stochastic, population-based optimization algorithm which uses vectors of real numbers as representations of solutions. A DE strategy, usually named with strings like

^{*} In WIVACE 2007: Workshop Italiano in Vita Artificiale e Computazione Evolutiva, 5-7 Settembre, Sampieri (RG)

$DE/x/y/z$, consists in using $y \in \mathbb{N}$ vector differences weighed by a parameter F to perturb the genotype of the individuals denoted with a string x (*best* stands for the best individual, *rand* for a randomly chosen one, *rand-to-best* for a random one, but it participates in the perturbation too). By employing components of existing population members to construct trial vectors, the crossover method, marked as z (*exp* = exponential, *bin* = binomial), efficiently shuffles information about successful combinations, enabling the search for an optimum to focus on the most promising area of solution space.

2 Experimental Results

The efficacy of CTRNN training by DE has been tested on a sample case: sequence generator task. The aim is to find a neural network capable of changing its state when an external trigger is given. By choosing to use only 3 neurons, our CTRNN can be studied as a continuous dynamical system described by the following system of 3 non-linear differential equations:

$$\tau_i \dot{y}_i = -y_i + \sum_{j=1}^3 w_{ij} \sigma(y_j - \theta_j) + w_i I, \quad \text{for } i = 1, 2, 3 \quad (1)$$

where I is the value of the trigger (it holds 0 when it is *OFF* and 1 when it is *ON*) and $\sigma(x)$ is the standard logistic activation function. If we fix $\theta_j = 0$ and $\tau_i = 1 \forall i, j$, then we have 12 weights to determine. The learning is made in a reinforcement way. We generate a random sequence $\mathbf{x}' = [bit^1, \dots, bit^M]$, where M , whose value has been extracted randomly, is the length of the sequence and $bit^i \in \{0, 1\} \forall i \in M$. For every sequence $\mathbf{x}'$ we generate the target $\mathbf{t}' = \{t^1, \dots, t^M\}$ by a procedure not reported here for the sake of brevity. Given the candidate vector $\mathbf{w}$ of the parameters, we evaluate how much the output $\mathbf{y} = \{\bar{y}_3^1, \dots, \bar{y}_3^M\}$ is similar to $\mathbf{t}'$ by a fitness function F_{SG} made up of two terms:

$$F_{SG}(\mathbf{y}(\mathbf{w})) = F_{HM}(\mathbf{y}(\mathbf{w})) + k \cdot F_{HD}(\mathbf{y}(\mathbf{w}))$$

where F_{HM} is the Hamming distance between the two strings and measures *how many times* the output of the network gives a fixed point value different from the expected target $\mathbf{t}$, and F_{HD} measures *how different* the fixed point values are from the desired targets. For the simulation we have chosen a $k = 0.01$.

We adopted a *DE/best/2/bin* strategy to discover the vector $\mathbf{w}$. Via an experimental trial, we assigned $F = 0.5$ and $CR = 0.7$ and we set the number of population members to 30 and the maximum generation number to 2500. The initial range of $\mathbf{w}$ components is $(-100, 100)$. Experimental results show that, in any of the 10 runs, DE picked out as solutions vectors $\mathbf{w}$ able to train the CTRNN correctly (for an example, see left side in Figure 1). Let us note two items. Firstly, the weights found are very sparse: different reached solutions go from values within the start range (e.g. $w_{21} \approx 21.19$) to values very far from it (e.g. $w_{13} \approx 1.86 \cdot 10^{18}$). In this case, in theory, the only limitation is the

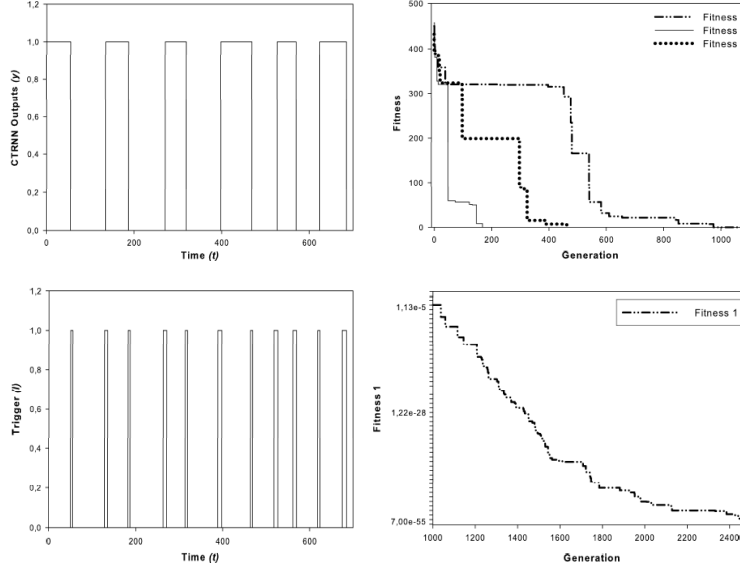


Fig. 1. Left Bottom: Trigger I as input. **Left Top:** A learned CTRNN output y . **Right Top:** Fitness plots of three runs as a function of the generation. **Right Bottom:** Fitness 1 from 1000-th to 2500-th generation.

precision of the machine on which the algorithm is running. That is an evidence of surmounting the boundedness and granularity problems. Secondly, the fitness landscape is very rough. So the descent of the fitness function towards the global minimum occurs in “steps” (see **Right Top** of Figure 1). Each run passes through a different sequence of local minima, from which the DE algorithm has to escape. The sparseness of the found values suggests that DE is almost able to investigate the entire parameter space in spite of the particular structure of the fitness landscape. Moreover, **Right Bottom** of Figure 1 illustrates how the search of the parameters continues even in the very proximity of optimal values. This issue seems to confirm DE efficacy in solving the blocking problem.

3 Conclusions

We presented a DE approach to CTRNN parameter learning. In this studied sample case, the sequence generator task, experimental results show that DE seems to be able to solve granularity, boundedness and blocking problems in an effective way. Besides, we have suggested that DE can be the optimal candidate for robotics applications, when trying to learn specific behaviours and also to model non-linear dynamical systems in general in a “CTRNN way”.

Next studies will concern a deeper investigation on theoretical details of DE approach for CTRNN learning and a direct comparison with rvGAs.